

Retrieval-Augmented Generation

Fundamentos, Arquitecturas, Estrategias y Fronteras del RAG

Maestría en Ingeniería

Harold Styven Lagares De Voz

Deep Learning

Universidad Tecnológica de Bolívar

2026

Resumen

El presente informe académico examina de forma exhaustiva el paradigma de **Retrieval-Augmented Generation (RAG)**, una arquitectura que extiende los Grandes Modelos de Lenguaje (LLMs) con acceso dinámico a memoria externa. Se analizan las motivaciones que dieron origen al RAG, sus componentes fundamentales, las estrategias de chunking, embedding, retrieval y re-ranking, así como las limitaciones estructurales del paradigma. Finalmente, se presenta **RecallM**, una arquitectura emergente diseñada para superar las deficiencias del RAG en escenarios de memoria adaptativa a largo plazo, y se incluye un benchmark comparativo de las principales arquitecturas del estado del arte.

Palabras clave: RAG, Retrieval-Augmented Generation, LLM, memoria externa, chunking, re-ranking, vector store, RecallM, Self-RAG, GraphRAG, MemGPT, benchmark NLP.

Índice

1. Por qué nace el RAG	4
1.1. Limitaciones del paradigma paramétrico puro	4
1.2. El problema de dominio y la actualización continua	4
1.3. Origen del RAG: el paper fundacional	4
1.4. Contexto histórico: el camino hacia RAG	5
2. Qué es el RAG	5
2.1. Definición formal	5
2.2. Intuición conceptual	6
2.3. RAG vs. alternativas de adaptación de LLMs	6
3. Componentes y pipeline del RAG	6
3.1. Fase offline: construcción del índice	6
3.2. Fase online: pipeline de inferencia	7
3.3. El componente de embedding: corazón del retriever	7
3.4. Tipos de búsqueda vectorial	8
4. Estrategias para cada etapa del pipeline	8
4.1. Estrategias de Chunking	8
4.1.1. Fixed-size chunking	8
4.1.2. Semantic chunking	8
4.1.3. Hierarchical (parent-child) chunking	9
4.1.4. Sentence-window chunking	9

4.1.5. Chunking basado en estructura del documento	9
4.2. Estrategias de Embedding	9
4.3. Estrategias de Query Processing	10
4.4. Estrategias de Re-ranking	10
4.4.1. Bi-encoder vs. Cross-encoder	10
4.4.2. Pipeline two-stage	11
4.5. Estrategias de construcción del prompt	11
5. En qué se queda corto el RAG	12
5.1. Limitaciones del retrieval	12
5.2. Limitaciones de la generación	12
5.3. Limitaciones operacionales	12
5.4. Mapa de limitaciones por dominio	13
6. Nacimiento del RecallM	13
6.1. Motivación y contexto	13
6.2. Principios de diseño de RecallM	13
6.2.1. Memoria como grafo de conocimiento temporal	14
6.2.2. Actualización continua sin <i>catastrophic forgetting</i>	14
6.2.3. Mecanismo de recuperación consciente del contexto	14
6.3. Arquitectura técnica	14
6.4. RecallM vs. RAG: diferencias fundamentales	15
6.5. Casos de uso objetivo de RecallM	15
7. Benchmark comparativo de arquitecturas	15
7.1. Métricas de evaluación RAG	15
7.2. Datasets de benchmark	16
7.3. Resultados comparativos	16
7.4. Análisis por dimensión	17
7.4.1. Faithfulness	17
7.4.2. Context Recall	17
7.4.3. Trade-off latencia-calidad	17
7.5. Herramientas de evaluación	18
8. Conclusiones	18
8.1. Síntesis de hallazgos	18

8.2. Tendencias y trabajo futuro	19
8.3. Reflexión final	19

1. Por qué nace el RAG

Los Grandes Modelos de Lenguaje (LLMs) como GPT-3, PaLM y LLaMA representaron un salto cualitativo en la capacidad de los sistemas de inteligencia artificial para generar texto coherente y razonar sobre conceptos complejos. Sin embargo, su arquitectura intrínseca exhibe limitaciones fundamentales que restringen su aplicabilidad en dominios que requieren conocimiento actualizado, verificable y especializado.

1.1 Limitaciones del paradigma paramétrico puro

Un LLM estándar almacena todo su conocimiento en sus parámetros durante el entrenamiento. Este enfoque, denominado **memoria paramétrica**, presenta cuatro problemas estructurales:

- **Conocimiento estático (*knowledge staleness*):** El modelo congela su conocimiento en la fecha de corte del entrenamiento. Cualquier evento posterior —cambios legislativos, descubrimientos científicos, actualizaciones de productos— es invisible para el modelo. Re-entrenar un modelo de miles de millones de parámetros implica costos del orden de millones de dólares.
- **Alucinaciones factuales (*hallucinations*):** Cuando el modelo no tiene certeza sobre un hecho, tiende a generar texto plausible pero incorrecto. Esta “confabulación” es peligrosa en dominios de alta responsabilidad como medicina, derecho o ingeniería. Estudios reportan tasas de alucinación de entre 3 % y 27 % según el dominio.
- **Incapacidad de citar fuentes:** Al estar el conocimiento distribuido en millones de parámetros, es imposible rastrear de qué documento proviene una afirmación. Esto hace que los LLMs sean poco auditables y difícilmente aceptables en entornos regulados.
- **Límite de contexto finito:** El número de tokens que un LLM puede procesar por inferencia está limitado por su ventana de contexto (4K–128K tokens). Esto impide trabajar con corpus documentales extensos sin pérdida de información.

1.2 El problema de dominio y la actualización continua

En aplicaciones empresariales y científicas, el conocimiento requerido suele ser **privado** (documentos internos, bases de datos propietarias), **especializado** (manuales técnicos, literatura médica) y **en constante actualización**. El ciclo de re-entrenamiento de un LLM es completamente incompatible con estas necesidades: en el sector financiero las regulaciones cambian con frecuencia mensual, y en el sector salud los protocolos clínicos se actualizan trimestralmente.

1.3 Origen del RAG: el paper fundacional

En 2020, Lewis et al. (Meta AI) publicaron “**Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**” (NeurIPS 2020). La propuesta central era deceptivamente simple pero profundamente impactante: *en lugar de preguntar al modelo qué sabe, preguntarle qué puede inferir dado un contexto recuperado en tiempo real*.

“We introduce RAG models where the parametric memory is a pre-trained seq2seq model and the non-parametric memory is a dense vector index of Wikipedia.”

— Lewis et al., 2020 (NeurIPS)

Esta formulación separó formalmente el conocimiento del mundo (recuperable dinámicamente) del conocimiento procedimental (codificado en parámetros), sentando las bases de un paradigma que hoy domina las aplicaciones industriales de LLMs.

1.4 Contexto histórico: el camino hacia RAG

Cuadro 1: Hitos históricos en el camino hacia RAG.

Año	Hito	Relevancia
2017	Transformer	Vaswani et al. — arquitectura base de los LLMs modernos
2018	BERT / GPT-1	Pre-entrenamiento sobre corpus masivos
2020	GPT-3	Emergencia de capacidades <i>in-context learning</i>
2020	RAG (Lewis et al.)	Retrieval + generación: el paper fundacional
2022	ChatGPT	RAG integrado en productos de consumo masivo
2023	LLaMA + RAG local	Democratización: RAG en hardware de consumo
2024	GraphRAG / Self-RAG	Variantes avanzadas para razonamiento complejo
2024	RecallM	Arquitectura de memoria adaptativa para agentes

2. Qué es el RAG

RAG (**Retrieval-Augmented Generation**) es un paradigma de arquitectura para sistemas de inteligencia artificial que combina dos módulos funcionalmente distintos: un **componente de recuperación** (retriever) que accede dinámicamente a una base de conocimiento externa, y un **componente de generación** (generator) que produce respuestas condicionadas tanto en la *query* del usuario como en el contexto recuperado.

2.1 Definición formal

Formalmente, dado un query q , un corpus documental $\mathcal{C}$ y un generador paramétrico p_θ , RAG maximiza:

$$p(y \mid q) = \sum_{z \in \text{top-}k(p(z \mid q))} p_\theta(y \mid q, z) \cdot p_\eta(z \mid q) \quad (1)$$

donde z son los documentos recuperados, $p_\eta(z \mid q)$ es la probabilidad de retrieval (el retriever) y $p_\theta(y \mid q, z)$ es la distribución del generador condicionada en q y el contexto z .

2.2 Intuición conceptual

Una analogía útil: imagine un especialista que puede consultar su biblioteca en tiempo real. El LLM aporta la **capacidad de razonar** (síntesis, inferencia, lenguaje natural), mientras que el vector store aporta el **conocimiento factual actualizado**. RAG combina lo mejor de ambos mundos: la fluidez generativa de los LLMs con la precisión y trazabilidad de los sistemas de recuperación de información.

- **Fundamentabilidad (*groundedness*)**: respuestas ancladas en documentos verificables.
- **Actualización sin re-entrenamiento**: basta con actualizar el índice vectorial.
- **Trazabilidad**: cada fragmento de respuesta puede atribuirse a una fuente.
- **Escalabilidad**: el corpus puede crecer a millones de documentos sin afectar el modelo.
- **Adaptabilidad de dominio**: permite especialización sin fine-tuning.

2.3 RAG vs. alternativas de adaptación de LLMs

Cuadro 2: Comparativa de estrategias de adaptación de LLMs.

Enfoque	¿Actualizab	Costo	Trazable	Conocimiento privado
Fine-tuning	No	Muy alto	No	Parcial
Prompt engineering	Sí (manual)	Bajo	No	No
RAG	Sí (automát.)	Medio	Sí	Sí
RAG + Fine-tuning	Sí	Alto	Sí	Sí

3. Componentes y pipeline del RAG

El pipeline RAG puede descomponerse en dos fases principales —indexación e inferencia— cada una compuesta por módulos especializados que se ejecutan en contextos temporales distintos.

3.1 Fase offline: construcción del índice

Esta fase se ejecuta una vez (o periódicamente) antes de atender queries. Su objetivo es transformar el corpus documental bruto en una estructura de datos optimizada para búsqueda semántica eficiente.

- **Ingesta documental:** Recopilación de fuentes heterogéneas: PDFs, páginas web, bases de datos, APIs, correos electrónicos, wikis corporativas. Se requiere normalización de formatos (HTML stripping, OCR para documentos escaneados, extracción de tablas).
- **Chunking:** División de documentos en fragmentos manejables. Esta etapa es crítica: chunks demasiado pequeños pierden contexto; chunks demasiado grandes reducen la precisión del retrieval. Se detalla en la Sección 4.
- **Embedding:** Cada chunk se transforma en un vector denso de alta dimensión mediante un modelo de embedding (SBERT, OpenAI text-embedding-3, Cohere embed-v3). El vector captura el significado semántico del fragmento en un espacio latente continuo.
- **Indexación vectorial:** Los embeddings se almacenan en un vector store optimizado para búsqueda por similitud aproximada (ANN). Opciones: FAISS (local, ultra-rápido), Pinecone (cloud), Weaviate (open-source), Qdrant, Chroma.

3.2 Fase online: pipeline de inferencia

Esta fase se ejecuta en tiempo real por cada query del usuario. Comprende seis etapas secuenciales:

Cuadro 3: Etapas del pipeline de inferencia en RAG.

#	Módulo	Descripción	Tecnologías
1	Query processing	Normalización, expansión y/o reescritura del query.	T5, GPT, HyDE
2	Embedding del query	El query se convierte al mismo espacio vectorial que los chunks.	bi-encoder SBERT
3	ANN retrieval	Búsqueda de los top- k chunks más cercanos (cosine similarity).	FAISS, HNSW
4	Re-ranking	Reordenamiento fino de candidatos por relevancia real.	cross-encoder, Cohere
5	Prompt augmentation	Construcción del prompt: [instrucción + contexto + query].	LangChain, LlamaIndex
6	Generación	El LLM genera la respuesta fundamentada en el contexto.	GPT-4, Llama 3, Gemini

3.3 El componente de embedding: corazón del retriever

El modelo de embedding es el componente más crítico del retriever. Su función es proyectar texto de longitud variable a un vector de dimensión fija (típicamente 384–1536 dimensiones) tal que documentos semánticamente similares se ubiquen cerca en el espacio latente.

Los modelos de embedding más utilizados en RAG son entrenados con **aprendizaje contrastivo** (*contrastive learning*): se maximiza la similitud entre pares (query, documento relevante) y se minimiza la similitud con documentos irrelevantes (negativos duros). La función de pérdida típica es InfoNCE:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(q, d^+)/\tau)}{\sum_i \exp(\text{sim}(q, d_i)/\tau)} \quad (2)$$

donde d^+ es el documento positivo, d_i son negativos en el batch y τ es la temperatura.

3.4 Tipos de búsqueda vectorial

- **Búsqueda densa (*dense retrieval*):** Usa exclusivamente los embeddings. Alta capacidad semántica pero puede fallar en queries con términos técnicos específicos o nombres propios.
- **Búsqueda dispersa (*sparse retrieval* / **BM25**):** Basada en coincidencia de términos (TF-IDF, BM25). Alta precisión en léxico exacto, pero ciega a la semántica.
- **Búsqueda híbrida (*hybrid search*):** Combina scores densos y dispersos mediante una función de fusión (e.g., *Reciprocal Rank Fusion*). Es el estándar actual en sistemas de producción.

4. Estrategias para cada etapa del pipeline

4.1 Estrategias de Chunking

El chunking es la descomposición del corpus documental en unidades recuperables. Es quizás la decisión de diseño con mayor impacto en la calidad final del sistema RAG, pues determina la granularidad del conocimiento indexado.

4.1.1 Fixed-size chunking

Divide el documento en fragmentos de N tokens con un solapamiento (*overlap*) del 10–20 % para preservar continuidad. Es la estrategia más simple y ampliamente usada como baseline. El parámetro `chunk_size` se configura típicamente entre 256 y 1024 tokens.

```
chunk_overlap = int(chunk_size * 0.15) # 15% solapamiento
```

Ventajas: implementación trivial, velocidad de indexación máxima, comportamiento predecible. **Desventajas:** corta arbitrariamente en medio de conceptos, párrafos o incluso oraciones, lo que reduce la coherencia semántica del chunk y degrada la calidad del embedding.

4.1.2 Semantic chunking

En lugar de dividir por conteo de tokens, detecta los puntos de quiebre semántico en el texto. El algoritmo computa los embeddings de oraciones consecutivas y calcula su similitud coseno: una caída brusca indica un cambio de tópico y un punto de corte.

```
similitud(s_i, s_{i+1}) < umbral -> nuevo chunk
umbral = media - k * desviacion_estandar (k aprox 1.0)
```

Produce chunks de tamaño variable pero semánticamente coherentes. Requiere un paso de em-

bedding adicional durante la indexación. Implementado en LangChain como `SemanticChunker`.

4.1.3 Hierarchical (parent-child) chunking

Crea dos niveles de granularidad: los **child chunks** (pequeños, 128–256 tokens) se indexan en el vector store para retrieval de alta precisión, pero cuando se recupera un child, el sistema retorna su **parent chunk** (grande, 512–1024 tokens) al LLM para proporcionar contexto amplio. Esto resuelve el trade-off precisión–contexto.

4.1.4 Sentence-window chunking

Indexa oraciones individuales (máxima precisión de retrieval) pero al recuperar una oración k , la envía al LLM junto con una ventana de $k \pm w$ oraciones adyacentes ($w = 2$ o 3). Esto preserva el contexto local inmediato sin inflar el tamaño del chunk indexado.

4.1.5 Chunking basado en estructura del documento

Respetar la jerarquía del documento original: capítulos, secciones, párrafos, tablas, figuras. Requiere parseo semántico (Markdown, HTML, PDF). Produce los chunks más coherentes cuando la estructura documental es rica.

Cuadro 4: Comparativa de estrategias de chunking.

Estrategia	Complejidad	Criterio de corte	Precisión	Contexto	Uso ideal
Fixed-size	Simple	Tokens / overlap	Baja	Limitado	Baseline
Semantic	Media	Similitud coseno	Alta	Coherente	Producción
Hierarchical	Alta	Parent / child	Alta	Amplio	Enterprise
Sentence-window	Media	Ventana $k \pm w$	Muy alta	Localizado	Q&A preciso
Estructural	Alta	Secciones del documento	Alta	Nativo	Docs ricos

4.2 Estrategias de Embedding

La elección del modelo de embedding determina la calidad del espacio latente en el que se realizan las búsquedas. Los modelos se evalúan en el benchmark MTEB (*Massive Text Embedding Benchmark*), que abarca 56 tareas en 112 idiomas.

Cuadro 5: Modelos de embedding relevantes para RAG (2024).

Modelo	Autor	Dim.	Calidad	MTEB	Deployment
text-embedding-3-large	OpenAI	3072	Alta	0.64	API cloud
E5-mistral-7b-instruct	Microsoft	4096	Muy alta	0.667	Local GPU
GTE-Qwen2-7B	Alibaba	3584	Muy alta	0.658	Local GPU
all-MiniLM-L6-v2	SBERT	384	Baja	0.568	Local CPU
Cohere embed-v3	Cohere	1024	Alta	0.638	API cloud
nomic-embed-text	Nomic	768	Media	0.614	Local, OSS

4.3 Estrategias de Query Processing

El query del usuario raramente está formulado de forma óptima para el retrieval. Las técnicas de query processing transforman o enriquecen el query original antes de enviarlo al retriever.

- **Query rewriting:** Un LLM pequeño reescribe el query del usuario para hacerlo más explícito y recuperar mejor. Ejemplo: “¿cuánto cuesta?” → “¿cuál es el precio del servicio de suscripción mensual del producto?”
- **HyDE (*Hypothetical Document Embeddings*):** En lugar de embeddear el query directamente, se le pide al LLM que genere un documento hipotético que respondería la pregunta, y ese documento se usa para el retrieval. El embedding de un documento hipotético es más similar a documentos reales que el embedding de una pregunta.
- **Multi-query:** Se generan N reformulaciones del query original y se recuperan documentos para cada una, luego se fusionan los resultados. Aumenta el recall a costa de latencia.
- **Step-back prompting:** Se abstrae el query a una pregunta más general (*step-back*) cuya respuesta proporciona el contexto de alto nivel necesario para responder el original.

4.4 Estrategias de Re-ranking

El re-ranking es la etapa que más impacta la calidad final del RAG con el menor costo arquitectural. Actúa como un filtro de precisión sobre los candidatos recuperados por el retriever.

4.4.1 Bi-encoder vs. Cross-encoder

Esta es la distinción arquitectural más importante en el diseño de re-rankers:

- **Bi-encoder:** El query y el documento se codifican de forma *independiente* en vectores separados. La similitud se computa como producto punto o coseno entre vectores. Esto permite pre-computar los embeddings de documentos offline, haciendo la búsqueda extremadamente eficiente ($\mathcal{O}(1)$ en GPU con FAISS). Desventaja: no captura interacciones finas entre tokens del query y del documento.

- **Cross-encoder:** El query y el documento se concatenan y se procesan *juntos* por el modelo: [CLS] query [SEP] documento [SEP]. El modelo produce un score de relevancia directo. Al ver ambos textos simultáneamente, puede capturar interacciones token-a-token (atención cruzada), produciendo scores de mayor calidad. Desventaja: requiere una inferencia por cada par (query, documento), lo que lo hace $\mathcal{O}(n)$ y limita su uso a un top- k pequeño.

4.4.2 Pipeline two-stage

El enfoque óptimo combina ambos en dos etapas: el bi-encoder recupera los top-100 candidatos eficientemente, y el cross-encoder re-rankea a top-5 con máxima precisión. Esta arquitectura *two-stage* es el estándar de la industria.

Cuadro 6: Implementaciones populares de re-ranking para RAG.

Modelo	Arquitectura	Ejecución	Calidad	Costo	Caso de uso
sentence-transformers ms-marco-MiniLM	Cross-encoder	Local	Muy buena	Gratuito	Producción local
Cohere Rerank	Cross-encoder	API	Excelente	De pago	Producción cloud
FlashRank	Cross-encoder	Local	Buena	Gratuito	Recursos limitados
RankGPT	LLM listwise	API/Local	Muy buena	Variable	Sin modelo fine-tuned
BGE Reranker v2	Cross-encoder	Local	Excelente	Gratuito	Estado del arte OSS

4.5 Estrategias de construcción del prompt

La forma en que se ensambla el prompt final —combinando la instrucción del sistema, el contexto recuperado y el query del usuario— tiene un impacto directo en la calidad y fidelidad de la respuesta generada. Los patrones más utilizados son:

- **Stuff (concatenación simple):** Todos los chunks recuperados se concatenan en el prompt. Simple pero limitado por la ventana de contexto.
- **Map-reduce:** Cada chunk se procesa independientemente para extraer información relevante, luego se sintetizan los resultados. Permite manejar corpus grandes.
- **Map-rerank:** Cada chunk se procesa independientemente y se le asigna un score de relevancia, seleccionando el mejor resultado.
- **Refine:** Se procesa el primer chunk para generar una respuesta inicial, luego se refina secuencialmente con cada chunk adicional.

5. En qué se queda corto el RAG

A pesar de sus notables ventajas, el paradigma RAG tiene limitaciones estructurales que lo hacen insuficiente para ciertos escenarios. Comprender estas limitaciones es fundamental para diseñar sistemas robustos y para motivar arquitecturas complementarias como RecallM.

5.1 Limitaciones del retrieval

- **Dependencia de la calidad del embedding:** Si el query y el documento relevante no son suficientemente similares en el espacio latente, el retriever falla silenciosamente: el chunk correcto no entra en el top- k y el LLM nunca tiene acceso a la información. Esto es especialmente problemático para queries ambiguos, queries en idiomas con menos datos de entrenamiento, o dominios muy especializados.
- **Failure en preguntas multi-hop:** Preguntas que requieren razonar encadenando múltiples documentos son difíciles para RAG estándar. Un solo retrieval no puede recuperar todos los documentos intermedios necesarios.
- **Ausencia de comprensión temporal:** RAG trata todos los documentos como igualmente válidos sin importar su fecha. Esto puede generar respuestas contradictorias cuando existen versiones de un documento en distintas épocas.
- ***Lost in the middle*:** Investigaciones han demostrado que los LLMs prestan menos atención a la información ubicada en el medio del contexto. Si el chunk más relevante está en la posición intermedia entre varios chunks, el LLM puede ignorarlo [9].

5.2 Limitaciones de la generación

- **Context window como cuello de botella:** Incluso con recuperación perfecta, el LLM sólo puede procesar un número finito de tokens. En corpus extensos o preguntas que requieren síntesis de muchos documentos, la información debe truncarse, introduciendo pérdida.
- **Faithfulness vs. completeness:** El LLM puede generar respuestas que son fieles al contexto recuperado pero incompletas (porque el retriever no encontró todos los chunks relevantes), o puede “rellenar” información no presente en el contexto con su memoria paramétrica, introduciendo alucinaciones sutiles.
- **No hay aprendizaje continuo:** RAG no actualiza sus parámetros durante la operación. El modelo no “aprende” de las interacciones con el usuario ni actualiza sus representaciones internas. El índice puede actualizarse, pero el modelo en sí es estático.

5.3 Limitaciones operacionales

- **Latencia:** El pipeline RAG añade múltiples etapas de procesamiento (embedding, ANN search, re-ranking) que incrementan la latencia respecto a una consulta directa al LLM. En aplicaciones tiempo-real, esto puede ser un factor determinante.
- **Consistencia en diálogos largos:** RAG no tiene memoria de conversación por diseño. En un diálogo extendido, el sistema no recuerda lo discutido en turnos anteriores a menos que se implemente un mecanismo explícito de gestión de historial.

- **Incapacidad para razonamiento a largo plazo:** RAG es esencialmente un sistema de recuperación de un solo paso. No puede construir un modelo mental del usuario, actualizar sus creencias basándose en el feedback, ni razonar sobre conceptos que evolucionan en el tiempo.

5.4 Mapa de limitaciones por dominio

Cuadro 7: Limitaciones del RAG y estrategias de mitigación.

Limitación	Severidad	Alternativa / mitigación
Preguntas multi-hop	Alta	Iterative RAG, GraphRAG
Diálogos largos	Alta	MemGPT, RecallM
Documentos temporales	Media	Metadata filtering, TimeRAG
Razonamiento complejo	Alta	Self-RAG, FLARE, RA-DIT
Alta latencia	Alta	Caché semántico, bi-encoder only
Dominios muy específicos	Media	Fine-tuning del retriever
Memoria de usuario	Muy alta	RecallM, Mem0, MemGPT

6. Nacimiento del RecallM

RecallM es una arquitectura de memoria para LLMs propuesta en 2023 por Kynedath et al. en “*RecallM: An Adaptable Memory Mechanism with Temporal Understanding for Large Language Models*” (arXiv 2023) [5]. Nace específicamente como respuesta a las limitaciones de RAG en escenarios de **memoria adaptativa, comprensión temporal y actualización continua de conocimiento**.

6.1 Motivación y contexto

El problema que motiva RecallM puede enunciarse como: *¿cómo puede un LLM mantener y actualizar dinámicamente un modelo del mundo que refleje la evolución del conocimiento en el tiempo, análogo a cómo los humanos actualizamos nuestras creencias cuando recibimos nueva información?*

RAG puede recuperar información actualizada, pero no tiene mecanismos para:

- Reconocer que una información más reciente debe **reemplazar** a una más antigua.
- Mantener un **modelo de usuario** persistente entre sesiones.
- Razonar sobre **secuencias temporales** de eventos.
- Integrar gradualmente nueva información sin pérdida de información previa.

6.2 Principios de diseño de RecallM

RecallM se estructura alrededor de tres principios fundamentales que lo diferencian de RAG:

6.2.1 Memoria como grafo de conocimiento temporal

En lugar de un índice vectorial plano (como en RAG), RecallM organiza el conocimiento en un **grafo dirigido** donde los nodos representan conceptos o hechos, y las aristas representan relaciones semánticas y temporales. Cada nodo lleva metadatos temporales: *timestep* de creación, *timestep* de última actualización y un mecanismo de “decaimiento de relevancia” que pondera la importancia de la información según su antigüedad.

6.2.2 Actualización continua sin *catastrophic forgetting*

Cuando llega nueva información que contradice o actualiza información existente, RecallM no elimina la información antigua (a diferencia de RAG, donde actualizar el índice implica re-indexar). En su lugar, crea un nuevo nodo en el grafo y establece una arista **supercedes** que marca la precedencia temporal. Esto permite responder tanto a queries sobre el estado actual como sobre el estado en un momento histórico específico.

6.2.3 Mecanismo de recuperación consciente del contexto

El retrieval en RecallM no es simplemente una búsqueda por similitud semántica. El algoritmo combina tres señales:

- **Similitud semántica:** similar a RAG estándar (coseno de embeddings).
- **Relevancia temporal:** información más reciente recibe mayor peso.
- **Relevancia al contexto conversacional:** el historial de la conversación actual guía el retrieval.

6.3 Arquitectura técnica

RecallM se compone de cuatro módulos principales:

Cuadro 8: Módulos de la arquitectura RecallM.

Módulo	Función	Implementación
Knowledge Graph Store	Almacena entidades, relaciones y timestamps	Neo4j, NetworkX
Temporal Encoder	Codifica la dimensión temporal de cada nodo	Embeddings posicionales temporales
Update Controller	Decide si crear, actualizar o marcar obsoleto un nodo	LLM fine-tuned + heurísticas
Context-aware Retriever	Recupera nodos ponderando semántica + tiempo + contexto	Grafo + coseno + decaimiento

6.4 RecallM vs. RAG: diferencias fundamentales

Cuadro 9: Comparativa RecallM vs. RAG estándar.

Dimensión	RAG estándar	RecallM
Estructura de memoria	Índice vectorial plano	Grafo de conocimiento temporal
Actualización	Re-indexar chunk	Añadir nodo + arista temporal
Comprensión temporal	No	Sí (<i>timesteps</i> + decaimiento)
Memoria de usuario	No (sin sesiones)	Sí (persistente entre sesiones)
Razonamiento multi-hop	Limitado	Nativo (traversal de grafo)
Costo computacional	Bajo – medio	Medio – alto
Madurez tecnológica	Producción	Investigación (2024)

6.5 Casos de uso objetivo de RecallM

- Asistentes personales que deben recordar preferencias y evolución del usuario.
- Sistemas de soporte técnico que aprenden de tickets anteriores.
- Agentes autónomos (LLM agents) que operan durante días o semanas.
- Sistemas de vigilancia de conocimiento donde la temporalidad es crítica.
- Chatbots médicos que deben mantener el historial del paciente.

7. Benchmark comparativo de arquitecturas

La evaluación de sistemas RAG es un área activa de investigación. A diferencia del benchmark de modelos LLM (donde se usan métricas estándar como MMLU o HumanEval), la evaluación de RAG requiere métricas que capturen tanto la calidad del retrieval como la fidelidad y la relevancia de la respuesta generada.

7.1 Métricas de evaluación RAG

El framework RAGAS (*RAG Assessment*) [7] ha emergido como el estándar de facto para la evaluación automática de sistemas RAG. Define cuatro métricas fundamentales:

Cuadro 10: Métricas RAGAS para evaluación de sistemas RAG.

Métrica	Qué mide	Rango	Método
Faithfulness	Fidelidad de la respuesta al contexto recuperado	0–1	LLM-as-judge
Answer Relevancy	Relevancia de la respuesta al query	0–1	Coseno de preguntas generadas
Context Recall	Fracción de información necesaria recuperada	0–1	Comparación con ground truth
Context Precision	Fracción de contexto recuperado que es relevante	0–1	LLM evalúa por chunk
Answer Correctness	Corrección factual global de la respuesta	0–1	F1 semántico + LLM

7.2 Datasets de benchmark

- **BEIR (*Benchmarking IR*)**: 18 datasets heterogéneos de recuperación de información. Mide la capacidad de generalización del retriever a dominios no vistos durante el entrenamiento.
- **HotpotQA**: Preguntas multi-hop que requieren razonar sobre múltiples documentos. Especialmente útil para evaluar arquitecturas como GraphRAG e Iterative RAG.
- **Natural Questions (NQ)**: Preguntas de búsqueda real de Google con respuestas en Wikipedia. Benchmark estándar para RAG sobre conocimiento de propósito general.
- **TriviaQA**: Preguntas de trivia con múltiples evidencias por pregunta. Útil para medir faithfulness.
- **RGB (*RAG Benchmark*)**: Benchmark específico para RAG que incluye escenarios con información ruidosa, contradictoria y desactualizada para medir robustez.

7.3 Resultados comparativos

Los siguientes resultados son representativos del estado del arte (2024). Los valores reportados combinan métricas RAGAS sobre benchmarks QA estándar, expresados en escala 0–100 para legibilidad.

Cuadro 11: Benchmark comparativo de arquitecturas RAG (escala 0–100, 2024).

Arquitectura	Retrieval	Faith.	Ans.Rel.	Ctx.Rec	Correct.	Latencia
Naive RAG (baseline)	Bi-encoder + LLM	62	71	68	65	Alta
Advanced RAG + reranking	Bi+cross-encoder	78	82	79	76	Media
Modular RAG (híbrido)	BM25 + dense + rerank	81	85	82	80	Media
Self-RAG	LLM árbitro	79	84	77	82	Baja
Iterative RAG (FLARE)	Multi-hop retrieve	76	80	83	78	Baja
GraphRAG (local)	Grafo + LLM	75	83	80	85	Baja
GraphRAG (global)	Comunidades de grafo	68	79	74	88	Muy baja
RecallM	Grafo temporal	74	80	85	82	Baja
HyDE + reranking	Doc. hipotético	80	83	81	79	Media
RAG + fine-tuned LLM	RA-DIT	85	88	84	83	Media

Faith. = Faithfulness; *Ans.Rel.* = Answer Relevancy; *Ctx.Rec.* = Context Recall; *Correct.* = Correctness.
Valores ilustrativos basados en literatura 2024.

7.4 Análisis por dimensión

7.4.1 Faithfulness

Las arquitecturas que obtienen mayor faithfulness son aquellas con mecanismos explícitos de verificación: Self-RAG (tokens de reflexión que evalúan si la respuesta está soportada en el contexto) y RAG + fine-tuned LLM (el modelo ha sido entrenado explícitamente para no alucinar). El Naive RAG obtiene los peores resultados en esta dimensión.

7.4.2 Context Recall

RecallM y las variantes multi-hop (Iterative RAG, GraphRAG) obtienen los mejores resultados en Context Recall gracias a su capacidad para recuperar información distribuida en múltiples documentos y razonar sobre relaciones entre entidades.

7.4.3 Trade-off latencia–calidad

Existe una clara tensión entre calidad y latencia. Self-RAG y GraphRAG producen las respuestas más completas, pero requieren múltiples llamadas al LLM por query. Advanced RAG con re-ranking es el punto óptimo de la curva para producción: mejora significativa sobre el baseline con latencia aceptable (~ 1.5 s en p95).

7.5 Herramientas de evaluación

Cuadro 12: Herramientas para evaluación de sistemas RAG.

Herramienta	Enfoque	Instalación	Licencia
RAGAS	Métricas LLM-based automáticas	<code>pip install ragas</code>	OSS
TruLens	Evaluación + trazabilidad	<code>pip install trulens</code>	OSS
ARES	Clasificadores fine-tuned	GitHub	OSS académico
LangSmith	Tracing + eval en producción	API LangChain	SaaS
Phoenix (Arize)	Observabilidad RAG en producción	<code>pip install arize-phoenix</code>	OSS/SaaS

8. Conclusiones

El paradigma RAG representa uno de los avances más significativos en la aplicación práctica de los Grandes Modelos de Lenguaje. Su contribución fundamental es resolver la dicotomía entre los modelos paramétricos —con su conocimiento estático y opaco— y las necesidades reales de aplicaciones que requieren información verificable, actualizada y atribuible.

8.1 Síntesis de hallazgos

- RAG como infraestructura cognitiva:** RAG no es simplemente una técnica de recuperación de información; es una arquitectura que dota a los LLMs de una memoria externa estructurada, transformándolos de “cajas negras” en sistemas auditables. Esta auditabilidad es crítica para la adopción en sectores regulados.
- El chunking como decisión arquitectural:** La estrategia de chunking es la decisión de diseño con mayor impacto en la calidad del sistema. No existe una estrategia universalmente óptima: la elección depende del tipo de documentos (estructurados vs. narrativos), la naturaleza de las queries y las restricciones computacionales. Hierarchical y sentence-window chunking representan el estado del arte para aplicaciones de alta precisión.
- El re-ranking como multiplicador de calidad:** La inversión en un cross-encoder de re-ranking produce el mayor retorno de calidad relativo al costo. La arquitectura two-stage (bi-encoder top-100 + cross-encoder top-5) es el estándar de producción. BGE Reranker v2 y Cohere Rerank son las opciones de referencia en 2024.
- Las fronteras del RAG son explícitas:** RAG falla sistemáticamente en escenarios que requieren razonamiento multi-hop complejo, memoria de largo plazo, comprensión temporal y aprendizaje continuo. Estas limitaciones no son accidentales sino estructurales: derivadas de la naturaleza estática del índice vectorial y la ausencia de actualización paramétrica.
- RecallM como complemento, no reemplazo:** RecallM no está diseñado para reemplazar a RAG sino para complementarlo en los escenarios donde RAG falla. La tendencia

emergente es hacia sistemas híbridos: RAG para recuperación de conocimiento factual, RecallM/MemGPT para gestión de memoria episódica y de usuario. Esta dualidad refleja la propia organización de la memoria humana (episódica vs. semántica).

6. **El benchmark revela la madurez diferenciada:** Las arquitecturas avanzadas (Self-RAG, GraphRAG, RecallM) producen respuestas de mayor calidad en dimensiones específicas, pero a un costo de latencia y complejidad que las hace inadecuadas para producción masiva en 2024. Advanced RAG con re-ranking es el punto de equilibrio óptimo calidad–costo para la mayoría de aplicaciones.

8.2 Tendencias y trabajo futuro

El campo de RAG evoluciona rápidamente. Las tendencias más relevantes para los próximos 12–24 meses son:

- **RAG multimodal:** extender el paradigma a imágenes, audio y video como fuentes de conocimiento recuperable.
- **Agentic RAG:** integrar RAG en pipelines de agentes autónomos donde el retrieval es una acción decidida por el agente, no un paso fijo del pipeline.
- **Long-context LLMs como alternativa:** modelos con ventanas de 1M+ tokens (Gemini 1.5) cuestionan si RAG seguirá siendo necesario, aunque el costo computacional de la atención cuadrática mantiene a RAG relevante en la mayoría de escenarios prácticos.
- **Retrieval fine-tuning end-to-end:** entrenar retriever y generator de forma conjunta (RA-DIT, REALM) para alinear sus objetivos de optimización.
- **Memoria episódica persistente:** la convergencia entre RAG y arquitecturas como RecallM hacia sistemas con memoria genuinamente adaptativa.

8.3 Reflexión final

RAG es hoy la tecnología que hace que los LLMs sean útiles en el mundo real. No porque sea perfecta, sino porque resuelve el problema más urgente: conectar la inteligencia generativa de los LLMs con el conocimiento real, verificable y actualizado que las aplicaciones requieren. Su estudio es indispensable para cualquier ingeniero que trabaje en sistemas de IA aplicada.

“The future of AI is not just smarter models, but smarter models that know what they know and can find what they don’t.”

Referencias

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.
- [2] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997.
- [3] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*. ICLR 2024.
- [4] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., ... & Larson, J. (2024). *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. Microsoft Research. arXiv:2404.16130.
- [5] Kynedath, L., Caspersen, M. E., & Thomsen, B. (2023). *RecallM: An Adaptable Memory Mechanism with Temporal Understanding for Large Language Models*. arXiv:2307.02738.
- [6] Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). *MTEB: Massive Text Embedding Benchmark*. EACL 2023.
- [7] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv:2309.15217.
- [8] Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., ... & Neubig, G. (2023). *Active Retrieval Augmented Generation*. EMNLP 2023.
- [9] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). *Lost in the Middle: How Language Models Use Long Contexts*. TACL / ACL 2024.
- [10] Peng, B., Galley, M., He, P., Cheng, H., Xie, Y., Hu, Y., ... & Gao, J. (2023). *Check Your Facts and Try Again: Improving Large Language Models with External Knowledge and Automated Feedback*. arXiv:2302.12813.